

Week 1B: Version Control with Git & GitHub

[Start Assignment](#)

- Due Sunday by 11:59pm
- Points 15
- Submitting a website url

Week 1B: Version Control with Git & GitHub

ADD 103: Introduction to Web Design



Learning Objective Alignment: Deploy (LO8) — "Publish and manage live websites using hosting platforms and version control."



Time: 30m Study / 60m Project |



Points: 15 |



Due: Sunday 11:59PM



Watch the Lesson

NOTE: The instructor in this video is **NOT** the one you will connect with for this course.

Your instructor is Jennie Johns, email: jjohns@mchenry.edu (or search for the GitHub username jjohns-mcc).

Lesson 1B Git and Github



Powered by Panopto



Lesson Textbook: Git & GitHub Demystified

i Print/PDF Note: If you need to print this page or download it as a PDF, please click and expand all collapsible details sections on this page **before** triggering the print command (`Ctrl + P` or `Cmd + P`) to ensure all content is visible.

▼ 1. What is Version Control?

Imagine you are writing a research paper. You save it as `paper.docx`. Then you make changes and save it as `paper_v2.docx`, then `paper_final.docx`, and finally `paper_final_v2_actualfinal.docx`. This is a messy, unreliable way to track history.

In software and web development, we use **Version Control Systems (VCS)** to manage changes. A version control system acts as a time machine for your code. It tracks every line of code you add, delete, or modify, and lets you save "checkpoints" (called commits). If

you make a mistake or break your layout, you don't have to hit "Undo" a hundred times; you simply restore the last working checkpoint.

▼ 2. The History of Git

In 2005, **Linus Torvalds** (the creator of the Linux operating system kernel) needed a new version control tool that was extremely fast, secure, and distributed. When existing commercial options failed to meet his standards, he sat down and wrote his own software over the course of a few weeks. He named it **Git** (which is British slang for an unpleasant or silly person—Linus jokingly noted he named all of his projects after himself).

Today, Git has become the absolute global industry standard. Nearly every software engineer, web developer, and UI designer in the world uses Git daily.

▼ 3. Git vs. GitHub: What's the Difference?

Beginners often confuse Git and GitHub. They are related, but they are not the same thing:

Git vs. GitHub Comparison

Git (Local Time Machine)	GitHub (Cloud Storage & Sharing)
• Local Software: It runs directly on your computer. • No Internet Needed: It tracks file changes and saves checkpoints locally. • Interface: Controlled via commands, VS Code, or desktop apps.	• Cloud Platform: A website and hosting service (owned by Microsoft). • Internet Required: It hosts your Git repositories online so you can share them. • Interface: Web portal, collaboration forums, and deployment servers.

Analogy: Think of **Git** as the camera on your phone that takes the photo (local). Think of **GitHub** as Instagram, where you upload the photos to share them with the world (cloud).

▼ 4. Commit Messages: Documenting Your Work

Every time you commit in Git, you are required to write a **commit message**. A commit message is a short text description explaining *why* you made a particular change.

Why is this important?

- **The Future You:** If you look at a file in three months, you won't remember why you deleted a styling property. Your commit history tells the story of your choices.
- **Collaboration:** In team environments, other developers read commit histories to understand the progress of the codebase.
- **Workflow Discipline:** Writing a clear message forces you to pause, review your changes, and summarize your work before checking it in.

Best Practices for Commit Messages:

- Keep it short and specific (under 50 characters).
- Start with an action verb (e.g. `Add about page and links`, `Fix header contrast`, `Restructure grid layout`).
- Never use generic messages like `fixed stuff`, `asdfasdf`, or `update`. Good developers write meaningful descriptions!

Installation & Setup Guide

Follow the steps below to download, configure, and connect Git and GitHub to your workstation. Expand each step to see instructions:

▼ Step 1: Install Git on Your Computer

Choose the directions below matching your operating system:



Windows PC Instructions

1. Go to the official download page: [Git for Windows Download Page](https://git-scm.com/download/win)  (<https://git-scm.com/download/win>).
2. Click **Click here to download** to grab the latest standalone installer.
3. Double-click the downloaded `.exe` file to run the installer.
4. Click **Next** on each screen to accept the default settings. When prompted to select the default editor, choose **Visual Studio Code**.
5. Click **Install**, then finish the setup.

macOS Instructions

1. Open your Mac's **Terminal** (Press `Cmd + Space`, type "Terminal", and hit Enter).
2. Type the command: `git --version` and press Enter.
3. If Git is not installed, macOS will display a pop-up prompt asking: *"The git command requires the command line developer tools. Would you like to install them now?"*
4. Click **Install** and accept the license terms. macOS will automatically download and install Git.
5. Alternatively, download the installer package from [Git for macOS Download Page](https://git-scm.com/download/mac)  (<https://git-scm.com/download/mac>).

▼ Step 2: Create a GitHub Account & Obtain the Student Developer Pack

1. Go to [GitHub Homepage](https://github.com)  (<https://github.com>).
2. Click **Sign up** at the top right. Follow the prompts to create your personal developer account.
3. **Required:** Register for the [GitHub Student Developer Pack](https://education.github.com/pack)  (<https://education.github.com/pack>) using your school `.edu` email address. This package is completely free but mandatory for this course.

 **Course AI & Academic Integrity Policy:** AI code generators (such as ChatGPT, GitHub Copilot, or Claude) may **NOT** be used to generate your HTML, CSS, or JavaScript code. All code submitted in this course must be hand-coded by you. **Permitted AI Uses:** You **MAY** use AI writing tools (such as Grammarly or Microsoft Word AI) to assist with

spellchecking, proofreading, and drafting the text copy for your pages, as well as AI image generators for graphical assets.

▼ Step 3: Open the ADD103 Folder and Configure Your Git Identity

Before making your first commit, you must introduce yourself to Git. Run these commands inside VS Code's integrated terminal:

Note: You may use AI to help you configure and/or troubleshoot Git and GitHub; I demonstrate this in the video.

1. Open VS Code. Go to **File > Open Folder...** and select your parent `ADD103` course directory. All work for this course must happen inside this folder!
2. Open the terminal in VS Code by selecting **Terminal > New Terminal** from the top menu (or press `Ctrl + ``).
3. Type the following command (replacing `Your Name` with your actual name) and press Enter:

```
git config --global user.name "Your Name"
```
4. Type the following command (replacing `your.email@example.com` with your actual email) and press Enter:

```
git config --global user.email "your.email@example.com"
```
5. Verify your settings by running: `git config --global --list`. You should see your name and email listed.

▼ Step 4: Initialize the Repository and Upload Your Playbook

Now, turn your local folder into a Git repository and upload your existing Developer Playbook page (created in Week 1A) to GitHub using VS Code's built-in tools:

1. Click on the **Source Control Icon** on the left-hand activity bar (looks like a branch with three nodes, or press `Ctrl + Shift + G`).

2. Click the blue **Initialize Repository** button at the top of the panel. This tells Git to start tracking changes inside your `ADD103` directory.
3. You will see your existing `week_01/index.html` file listed under "Changes" with a green "U" (meaning Untracked).
4. Type a meaningful commit message in the text box above: `Initial commit`.
5. Click the blue **Commit** button (or checkmark icon). This saves your first "checkpoint" locally.
6. Click the blue **Publish Branch** button that appears.
7. A dropdown prompt will appear at the top of VS Code. Select either **Publish to GitHub private repository** or **Publish to GitHub public repository**.

 **Privacy Choice:** You are welcome to keep your repository **Private** or make it **Public**. Regardless of which option you choose, you must complete the next step (adding your teacher as a collaborator) so the instructor has full access to grade your code.

8. VS Code will open a browser window asking you to authenticate with GitHub. Log in and click **Authorize VisualStudioCode**. Your repository is now live on GitHub!

▼ Step 5: Add Your Teacher as a Collaborator

You must grant the instructor access to view your repository settings and revision history. Follow these steps on GitHub.com:

1. Open your web browser and go to your repository homepage on **GitHub.com** (e.g., `https://github.com/[your-username]/ADD103`).
2. Click on the  **Settings** tab in the horizontal menu bar at the top of your repository page.
3. In the left-hand sidebar menu, click on **Collaborators** (located inside the "Access" section).
4. Confirm your GitHub password if prompted.
5. Click the green **Add people** button.
6. In the search box, enter the teacher's email address: `jjohns@mchenry.edu` (or search for the GitHub username jjohns-mcc).
1. This is NOT the same as the instructor's name in the video below.

7. Select the correct account from the dropdown list and click **Add to this repository**.



Project: Git & GitHub Reflective Notes

Now that your repository is set up, you will create a second page to write notes to yourself about the Git workflow. Expand each step below to follow the workflow:

i Working on multiple computers? If you ever need to clone your repository to another PC or laptop, check out the reference wiki page: [Cloning a Git Repository](https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository)  (<https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>).

▼ Step 1: Create the git.html Notes Page

1. Make sure VS Code is open to your `ADD103` parent folder.
2. Inside the `week_01` folder, create a brand-new file named exactly: `git.html` (all lowercase).
3. Open `git.html`, type `!`, and press **Tab** to scaffold the standard HTML5 skeleton.
4. Set the `<title>` tag content in the head to: `Git & GitHub notes`.
5. In the `<body>` of your document, build the following structure (you do not need to link pages together yet):
 - An `<h1>` tag containing: `Git and GitHub`
 - An `<h2>` tag containing: `1. Understanding Git and GitHub`
 - One or more paragraphs (`<p>`) describing Git and GitHub in your own words, and explaining how they relate to and differ from each other.
 - An `<h2>` tag containing: `2. VS Code Source Control Process`
 - One or more paragraphs (`<p>`) explaining your step-by-step process for using Source Control in VS Code to save changes and commit. Treat this as notes to yourself for how to upload your pages next time!

▼ Step 2: Commit and Push the Notes Page

1. Save your files (`Ctrl + S` or `Cmd + S`).
2. Go to the **Source Control** panel in VS Code.
3. You will see `git.html` listed under "Changes" with a green "U" (meaning Untracked).
4. Type a meaningful commit message in the text box above (e.g. `Add git notes page`).
5. Click the blue **Commit** button (or checkmark icon). This saves your checkpoint locally.
6. Click the blue **Sync Changes** (or Push) button to upload the commit to GitHub.
7. **Verify on GitHub:** Open your web browser, go to your repository on GitHub.com (e.g., `https://github.com/[your-username]/ADD103`), and verify that your `week\_01` folder now shows both `index.html` and `git.html` uploaded successfully.

▼ Step 3: Submit Your Project

Once you have pushed your files and verified they are online, submit the direct link of your repository on Canvas:

1. Copy the direct URL of your repository from your browser's address bar (e.g. `https://github.com/[your-username]/ADD103`).
2. Submit this direct link as your assignment submission.



Grading Rubric

Grading Rubric - Git & GitHub Setup

Criteria	Distinguished (Full)	Proficient (Partial)	Emerging (Minimal)
GitHub Repository Setup (5 pts)	An ADD103 repository (public or private) is created on GitHub with	Repository exists but folder	Empty repository or

Criteria	Distinguished (Full)	Proficient (Partial)	Emerging (Minimal)
	the week_01 folder containing both index.html and git.html successfully pushed online. (5 pts)	structure/files are capitalized incorrectly, or git.html is missing. (3 pts)	no repository created. (0 pts)
Collaborator Access (5 pts)	The teacher (mkasprak@mchenry.edu) is added as a collaborator to the repository, allowing access to inspect logs. (5 pts)	Collaborator invitation was sent but to the wrong username or email address. (3 pts)	No collaborator added to the repository. (0 pts)
VS Code Git Workflow (5 pts)	Git version control is initialized locally, user identity (name/email) is configured, and files are committed and pushed via the VS Code interface. (5 pts)	Commit messages are present but low quality (e.g. empty or random characters), or Git configuration is missing. (3 pts)	Files uploaded manually to GitHub without using VS Code Source Control interface. (0 pts)

© 2026 Dr. Meri Kasprak in collaboration with Gemini & Antigravity